写在前面：

本笔记主要是对CSAPP常见题目的梳理，纯应试。

（因为我是自动化学生，学习这个完全一头雾水…只能死记硬背）

2026春考试完全是对作业题的复制粘贴，gcl老师出的纯搬人的卷子。

湘云

2026.7

本人94.8/100. Rank 4/125.

第二章

1. [补码]

7. –1029 的 16 位补码用十六进制表示为

负数补码: 正数取反+1

$1029 = 1024+4+1 = 2^{10}+2^2+2^0$

$= (0000\ 0100\ 0000\ 0101)_2$

$\Rightarrow -1029 = (1111\ 1011\ 1111\ 1011)_2$

15 F 11 B 15 F 11 B

$= 0xFBFB$

2. [补码][类型转换]

```
short t=-8196;
int y=t;
```

y的十六进制表示?

short → 2bytes $\xrightarrow{\times 8}$ 16 bits (位)

int → 4bytes $\xrightarrow{\times 8}$ 32 bits

$8196 = 8192+4 = 2^{13}+2^2 = (0010\ 0000\ 0000\ 0100)_2$ (16位)

13=D 12=C

$\Rightarrow -8196 = (1101\ 1111\ 1111\ 1100)_2$

$= 0xDFFC$

short, int默认为有符号, 进行符号扩展:

- 符号扩展: 扩展第1位
- 无符号扩展: 扩展0.

$\Rightarrow y = 0xFFFFDFFC$

3. [位运算][清零字节]

写出将int变量x的最高有效字节清零的C表达式

int一共4字节，0xAB CD EF GH ⟶ 0x00 CD EF GH

根据：1&原位=原位，0&原位=0：设计掩码 0x00FF FF FF

C表达式：x&=0x00FFFFFF

4. [位运算][判断位]

2. 判断整型变量 n 的位 7 为 1 的 C 语言表达式是

掩码：0100 0000 除了7位以外全置0，7位为0时整个数都是0.

C表达式：n&0x40

5. [位运算][移位]

6. 设机器数字长 8 位（含 1 位符号位），若机器数 DAH 为补码，分别对其进行算术左移一位和算术右移一位，其结果分别为（ ）

十六进制后缀H

DAH = $(11011010)_2$

逻辑移位：补0

算术移位：低补0，高补发号

⇒算术左移：$(10110100)_2$ = 0xB4

右移：$(11101101)_2$ = 0xED

6. [正溢出、负溢出]

5. 给定字长的整数 x 和 y 按补码相加，和为 s，则发生正溢出的情况是

A. x>0,y>0,s≤0 B. x>0,y<0,s≤0

C. x>0,y<0,s≥0 D. x<0,y<0,s≥0

同理，负溢出⇒x,y均负，和变为非负数

7. [T/U转换]

12. 以下关系表达式，结果为“真”的是(　)。

A. 2147483647U > –2147483648

B. (unsigned) –1 > –2

C. –1 < 0U

D. 2147483647 < (int) 2147483648U

int范围：$-2^{31} \sim 2^{31}-1$

unsigned int范围：$0 \sim 2^{32}-1$

T　　U

线性映射：负数x $\xrightarrow{T2U}$ $x+2^{32}$

比较规则：优先转为无符号.（有符号与无符号比较时）

A. 左 $=2^{31}-1$

右转为无符号：$-2^{31}+2^{32}=2^{31}$

∴左<右. Ax

B. 左 $=2^{32}-1$

右 $=2^{32}-2$. ✓

C. 左 $=2^{32}-1$，右 $=0$　　X

D. 两边都是有符号：2^{31} $\xrightarrow{\text{转为有符号}}$ $2^{31}-2^{32}=-2^{31}$

$2^{31}-1 > -2^{31}$.　　Dx

8. [强制类型转换]

17. 考虑以下 C 语言代码：

```
short si = -8196;
unsigned short usi = si;
```

执行上述程序段后，usi 的值是（　）。

两种思考方向.

① T→U：usi $=2^{16}-8196=57340$

② 重新解读：-8176 补码 $0xDFFC=(1101111111111100)_2$

$=2^{15}+2^{14}+2^{12}+2^{11}+\cdots+2^2=57340$

9.[小端存储]

5．64 位系统中，整型变量 x = -7，其在内存从低地址到高地址依次存放的数是 _____(十六进制表示，小端模式)。

整型变量一般指 int.

$7 = 0x00000007$

$-7 = 0xFFFFFFF9$

④ ③ ② ① ← 小端

$7 = (0111)_2$

取反+1：$(1001)_2$

$\Rightarrow$ 0xF9 0xFF 0xFF 0xFF

10.[IEEE 754单精度浮点数]

将 -1024 采用 IEEE754 单精度浮点数格式表示.

Step1：拆为 $(-1)^S \times 1.frac \times 2^{exp-127}$

$-1024 = (-1)^1 \times 1.0 \times 2^{10}$

$\Rightarrow S=1,\ frac=0,\ exp=137=(10001001)_2$

Step2：拼：S | exp | frac（23位0）

1 10001001 00---0

转十六进制：0xC4800000

11.[IEEE 754单精度浮点数][舍入]

1. 请说明 float 类型编码格式，并按步骤计算十进制-10.1 的各部分内容，写出十进制-10.1 在内存从低地址到高地址的存储字节内容（小端系统）。

$-10.1 = (-1010.0001100110\cdots)_2$

$= (-1)^1 \times 1.01000011001100\cdots \times 2^3$

$\therefore S=1,\ frac=01000011001100\cdots,\ S=3+127=130=(10000010)_2$

（取2位）

$= 01000011001100110011001$ 下一位是1，且后面还有1，进位.

编码：1 10000010 01000011001100110011010

C 1 2 1 9 9 9 A

小端序：0x9A 0x99 0x21 0xC1

12. [IEEE 754解码]

	S	exp	frac
①±0：		全0	全0
②非规格化		全0	非0
③规格化		除0除1	
④±∞		全1	全0
⑤NaN		全1	非0

非规格化数：exp全0.

$(-1)^S \times 0.frac \times 2^{-126}$

13. 假定某数采用 IEEE 754 单精度浮点数格式表示为 00000001H，则该数的值是

S=0, exp全0, frac非0 → 非规格化.

$(-1)^S \times 0.frac \times 2^{-126} = (0.\overbrace{00\cdots01}^{23\text{个}})_2 \times 2^{-126}$

$= 2^{-23} \times 2^{-126} = 2^{-149}$

13. [海湾战争舍入误差]

2. 1991 年 2 月 25 日，海湾战争中，美国在沙特阿拉伯的达摩地区设置的爱国者导弹拦截伊拉克的飞毛腿导弹失败，致使飞毛腿导弹击中了美军的一个兵营，造成 28 名士兵死亡。这是由一个浮点数的精度问题造成的。爱国者导弹系统中有一个内置时钟，用计数器实现，每隔 0.1s 计数一次。程序用 0.1 乘以计数器的值得到以秒为单位的时间。0.1 的二进制表示是一个无限循环序列：0.00011 [0011]…（方括弧中的序列是重复的）。请问：

（1）假定用一个类型为 float 的变量 x 来表示 0.1，则变量 x 在机器中的机器数是什么（要求写成十六进制形式）？绝对值 |x-0.1| 的值是什么（要求用十进制表示）？

（2）爱国者系统启动时计数器的初始值为 0，并开始持续计数。假定当时系统运行了 200h，则程序计算的时间和实际时间的偏差为多少？如果爱国者根据飞毛腿的速度乘以它被侦测到的时间来预测位置，若飞毛腿的速度为 2000m/s，则预测偏差的距离为多少？

(1) 运用IEEE表示：$(-1)^S \times 1.frac \times 2^{exp-127}$

$0.1 = (-1)^0 \times 1.1001\,1001\cdots \times 2^{-4}$

S=0, frac=1001 1001 1001 1001 1001 10|0 (舍入), exp=123

$=(0111\,1011)_2$

3 D C C C C C D

⇒ 0 011 1101 1100 1100 1100 1100 1100 1101

RPM=0x3DCCCCCD = 13421773.

$x = \frac{M}{2^{23}} \times 2^{-4}$?

$|x-0.1| = 1.49\times10^{-9}$

计数次数：$200\times60\times60\times\frac{1}{0.1s} = 7.2\times10^{6}$ 次

累计误差：0.010728s

距离误差：$0.010728\times2000m/s = 21.456m$

14.[浮点数和运算性质]

判断：

(√) 浮点数不是实数

(×) 浮点加法、乘法都满足结合律 都没有

是因为浮点数运算每一步都可能存在舍入.

(√) 浮点运算无分配律

15.[类型转换]

11．假定变量 i、f 的数据类型分别是 int、float。已知 i=12345，f=1.2345e3，则在一个 32 位机器中执行下列表达式时，结果为“假”的是(　)。

A. i==(int)(float)i

B. i==(int)(double)i

C. f==(float)(int)f

D. f==(float)(double)f

A. i很小，转float可精确表示，转回来还是12345

B. double 无损没问题

C. 转int时截断了小数部分 ×.

D. double 比 float 大，不会造成丢失.

16.[单精度浮点最大舍入误差]

18. 设某系统采用 IEEE 754 单精度浮点数格式，舍入模式为就近舍入（round to nearest）将一个实数转换为该浮点数时，由舍入引入的最大绝对误差为：

A. 2^{-24}　　B. 2^{-23}　　C. 2^{103}　　D. 2^{127}

√

可以直接记

17.[C语言除0行为]

9．C 程序执行到整数或浮点变量除以 0 可能发生(D)。

A. 显示除法溢出错直接退出 → 也有常见

B. 程序不提示任何错误 与异常设置有关

C. 可由用户程序确定处理办法

D. 以上都可能

浮点数 / 0 得到正/负无穷.

第三章

1. [调用约定]

整数/指针参数前6个 ⟶ %rdi, %rsi, %rdx, %rcx, %r8, %r9

返回值 ⟶ %rax

保存栈指针 ⟶ %rsp（栈向低地址增长）

4. 在 x86-64 系统中，调用函数 int gt (long x, long y)时，保存参数 y 的寄存器是(B)　rdi　rsi

A. %rdi B. %rsi C. %rax D. %rdx

5. 在 x86-64 系统中，调用函数 long gt (long x, long y)时，保存返回值的寄存器是(C)　rax

A. %rdi B. %rsi C. %rax D. %rbx

8. 在 X86-64 中，程序的栈存放在内存中，栈顶元素的地址使所有栈中元素中最 低 的，栈指针寄存器 %rsp 保存着栈顶元素的地址。

2. [操作数寻址]

D(Rb, Ri, S) ⇒ 地址: D + Rb + Ri * S （再取地址的内容）

- S只能为1, 2, 4, 8
- 带括号：将寄存器里的值视为地址，进行内存访问.
- $开头：立即数.

10. 已知内存和寄存器中的数值情况如下：

内存地址	值
0x100	0xff
0x104	0xAB
0x108	0x13
0x10c	0x11

寄存器	值
%rax	0x100
%rcx	0x1
%rdx	0x3

请填写下表，给出对应操作数的值：

操作数	值
%rax	0x100
(%rax)	0xff
9(%rax, %rdx)	0x11
0xfc(, %rcx, 4)	0xff
(%rax, %rdx, 4)	0x11

(2) 访问 0x100 地址，取地址中的值

(3) 9 + 0x100 + 0x3 = 0x10C. 访问该地址 ⟶ 0x11

(4) 0xfc + 4×0x1 = 0x100 访问该地址 ⟶ 0xff

(5) 0x100 + 4×(0x3) = 0x10C —访问该地址→ 0x11

3. [指令合法性]

4(%rsp)：内存，地址为%rsp+4

mov
- b → 1 bytes (%al, %bl, %cl, %dl, %sil, %dil, ...)
- w → 2 bytes (%ax, %bx, %cx, %dx, %si, %di ...)
- l → 4 bytes (大多e开头：%eax, %ebx, %ecx, %edx, %esi, %edi, ...)
- q → 8 bytes (大多r开头：%rax, %rbx, %rcx, %rdx, %rsi, %rdi, ...)

倒吃甘蔗

判断mov合法性
- 目的操作数不能是立即数
- 不能内存→内存
- 源和目的数据大小必须匹配指令后缀
- 立即数大小 < 指令指定大小

6. 下列传送指令中，哪一条是正确的（ ）
A. movb $0x105, (%ebx)
B. movl %rdi,%rax
C. movq %rdi,$105
D. movl 4(%rsp),%eax

取址一般用64位寄存器.

A. movb → 1byte (8位).
$0x105 有9位，不合法. ×

B. l → 32位，而%rdi，%rax是32位寄存器. ×

C. 目的操作数不能是立即数. ×

D. ✓

4. [数据传送与扩展]

12. 假设变量 sp 和 dp 被声明为类型：
Src_t *sp;
Dest_t *dp;
这里的 Src_t 和 Dest_t 是用 typedef 声明的数据类型，我们想使用适当的数据传送指令来实现下面的操作：
*dp = (Dest_t) *sp;
sp 和 dp 的值分别存储在%rdi 和%rsi 中，对下表的每个表项，请写出合适的两条传送指令（如需用到其他寄存器，使用%rax）。注意：规定如果强制类型转换既涉及大小变化又涉及符号变化时，操作应先改变大小。

sp指向的地址中读出一个Src_t类型的数，把其强转为Dest_t，存在dp指向的地址

Src_t	Dest_t	指令
char	int	①_____
char	unsigned	②_____
unsigned char	long	③_____
int	char	④_____
unsigned	unsigned char	⑤_____

5 2

数据传送与扩展

- 小→大：有符号用 movs…符号扩展，无符号用 movz…零扩展
- 大→小：直接截断低位

注意：当 %eax 会自动把 %rax 高32位清零

① char → int. 使用符号位扩展.

```
movsbl (%rdi), %eax
```

1字节　　符号扩展：32位

```
movl %eax, (%rsi)
```

→ 32位结果存在 dp 指向的位置

② char → unsigned (大小、符号都变. 位数表先改大小)

```
movsbl (%rdi), %eax
movl %eax, (%rsi)
```

③ unsigned char → long (8 bytes)

```
movzbq (%rdi), %rax
movq %rax, (%rsi)
```

④ int → char (1 bytes)

```
movl (%rdi), %eax
movb %al, (%rsi)
```

⑤ unsigned → unsigned char

```
movl (%rdi), %eax
movb %al, (%rsi)
```

大转小：直接截断

5.[数据传送与扩展]

2. 分析以下 x86-64 指令（4 分）

```
.varx
    int 123, -2345   # 123 的地址是 varx, -2345 在 varx+4

movq $varx, %rax
movl 4(%rax), %edx
movl $-1, %rax
```

(1) 执行 `movl 4(%rax), %edx` 后 `%rdx` =_____（long long int 十进制表示）

(2) 执行 `movl $-1, %rax` 后 `%rax` =_____（long long int 十进制表示）

错

第1步：$varx → %rax 把varx这一地址存入rax寄存器.

第2步：%rax+4所值 存入%edx
-2345

32位补码：$-2345 = 2^{32} - 2345$.

⇒ %rdx高32位已被清零，故%rdx存储：$2^{32} - 2345 = 4294964951$

第3步：-1存入%rax (64位)

⇒由于是movl，故实际为 movl $-1, %eax.
(4bytes, 32位)

$-1 = 0x00000000FFFFFFFF = 4294967295$

6. [条件码，条件传送]

11. 有下列C函数：

```
long max(long x, long y)
{                rdi   rsi
    long result;
    if(x >= y){
        result = x;
    }else{
        result = y;
    }
    return result;
}
```

1）请写出红色部分代码使用条件数据传输来实现条件分支的等价C语言形式。
2）给出对应的汇编指令，**要求使用条件传送指令**（初始执行指令 movq %rdi, %rax）。

cmp S,D：比较D-S，只设条件码

jl, jg, je, jne：看条件码跳转.

小于0 大于0 相等 不等
D<S D>S D=S D≠S
(有符号比较)

cmovl S,D：如果前面比较结果为"<"，则把S传送到D.
(less)

(1) result=(x>=y)?x:y;

(2) movq %rdi, %rax;
cmp %rsi, %rdi; //x-y
cmovl %rsi, %rax; //x-y<0则把y赋给返回值.

ret.

7.[结构体布局]

① 每个成员必须对齐在它自身对齐要求的倍数的偏移地址上.

② 结构体的总大小必须为最大对齐大小的整数倍.

7.C 语言程序定义了结构体 struct noname{char c; long n; int k; float *p; short a;};若该程序编译成 64 位可执行程序，则 sizeof(noname)的值是 40 。

char: 1
long: 8
int: 4
*float: 8 (指针)
short: 2

0-7 c 1
8-15 n 8
16-23 k 4
24-31 *p 8
32-39 a 2

8.[汇编代码改错]

3. 分析以下 x86-64 机器上的代码（6 分）

```
struct rec {
    int a[4];
    int i;
    struct rec* next;
}
void fun(struct rec* r, int val) {
    while (r) {
        int i = r->i;
        a[i] = val;
        r = r->next;
    }
}
```

rdi rsi

下面是实现上述循环的汇编代码，代码中存在错误，在错误的指令右边写出正确的汇编指令（注意：无需添加寄存器）。

```
jmp L12
.L11:
    movslq 16(%esi), %rax
    movl %esi, (%edi, %eax, 4)
    movq 20(%edi), %edi
.L12:
    testq %edi, %edi
    je L11
```

r==0?

test A,B ⇒ A&B==0?

```
jmp L12
.L11:
    movslq 16(%edi), %rax
    movl %esi, (%edi, %eax, 4)
    movq 24(%rdi), %rdi
.L12:
    testq %edi, %edi
    jne L11
```

// r≠0继续循环.

i (int32位)

结构体的存储验证：

a[0]: 0-3
a[1]: 4-7
a[2]: 8-11
a[3]: 12-15
i: 16-19 → r→i在r+16处
*next: 24-31 → r→next在r+24处

9.[缓冲溢出]

对下列程序，简述缓冲区溢出攻击的原理及防范方法.

```
int getbuf(char *s){
    char buf[32];
    strcpy(buf, s);
}.
```

原理：strcpy不检查长度. 若s超过32字节，会把buf后面的栈内容覆盖，从而改写程序原有的指令，数据.

防御方法：①代码中避免造出漏洞：使用限制字符串的库函数.

②随机栈偏移

③限制可执行代码的区域.

④进行栈破坏检查(金丝雀)

上述代码的改动例：

```
int getbuf_s(char *s){
    char buf[32];
    if (strlen(s)<sizeof(buf)){
        strcpy(buf,s);
    }
    return 0;
}
```

第四章.

*指令表

字节	0	1	2	3	4	5	6	7	8	9
halt	0	0								
nop	1	0								
cmovXX rA, rB	2	fn	rA	rB						
irmovq V, rB	3	0	F	rB	V					
rmmovq rA, D(rB)	4	0	rA	rB	D					
mrmovq D(rB), rA	5	0	rA	rB	D					
OPq rA, rB	6	fn	rA	rB						
jXX Dest	7	fn	Dest							
call Dest	8	0	Dest							
ret	9	0								
pushq rA	A	0	rA	F						
popq rA	B	0	rA	F						

注意这里的数字都是16进制，也就是一个数字占4bit，两个数字是1B。

注意：rrmovq rA, rB也是这种格式

0xF：操作数不是寄存器

F指的是R15的编码1111

或者就是16进制的F

23

停止 空指令 条件传送 传送 算术逻辑 跳转 子程序调用 子程序返回 入栈 出栈

1.[指令长度]

1. 分析以下 Y86-64 命令

```
0x006  irmovq $10,  %rsp
_____  call Label
_____  addq %rdx,  %rax
_____  ret
```

(1) 在表中填写空缺的指令地址。(3 分)

irmovq : 10 bytes

call: 9 bytes.

addq: (算术) 2 bytes

ret: 1 byte

① 0x006 + 0xA = 0x010

② 0x010 + 0x9 = 0x019

③ 0x019 + 0x2 = 0x01B

2.[条件跳转]

7. 在 Y86-64 架构的机器上，有下列汇编代码，请指出 jne t 指令之后执行的那条指令的地址为_____（顺序执行，不考虑流水线）。

```
0x000:     xorq %rax,%rax
0x002:     jne  t
...
0x019: t:  irmovq $3,  %rdx
0x023:     irmovq $4,  %rcx
0x02d:     irmovq $5,  %rdx
```

0x000: %rax 异或 %rax = 0，设ZF=1

0x002: if 不等于，跳转至t，但ZF=1，不跳转
(ZF=0)

故下一条: 0x002 + 0x9 = 0x00B
(jne → 9 bytes)

3. [时序/组合]

4. Y86-64 的顺序结构实现中，寄存器文件读时是作为时序逻辑器件看待

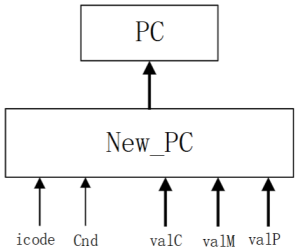
X.

{ 读 → 组合
  写 → 时序

4. [PC的更新逻辑]

9. 在 Y86 的 SEQ 实现中，PC（Program Counter，程序计数器）更新的逻辑结构如下图所示，请根据 HCL 描述为①②③④选择正确的数据来源。其中：Icode 为指令类型，Cnd 为条件是否成立，valC 表示指令中的常数值，valM 表示来自返回栈的数据，valP 表示 PC 自增。（C）

```
Int new_PC = [
    # Call
    Icode = ICALL: ①;
    # Taken branch.
    Icode = IJXX && Cnd: ②;
    # Completion of RET instruction.
    Icode = IRET: ③;
    # Default.
    1: ④;
```



A. valC, valM, valP, valP
B. valC, valC, valP, valP
C. valC, valC, valM, valP
D. valM, valC, valC, valP

更新PC {
- 函数调用: valC
- 分支跳转: valC
- 函数返回: valM
- 默认: valP.

→ 直接背.

5. [微动(下)].

→ 必须要记.

建议全背. → 考前记得默一下.

Tips: {
- R[ ]: 寄存器堆 (R[p]: 从p寄存器中读数据)
- M[ ]: 对地址进行1个字节的读写.

指令区

① OPq rA, rB.

eg: addq %rax, %rbx

编码: | 60 | rA rB | 2bytes.

① 取指: icode: ifun ← M_1[PC]

rA: rB ← M_1[PC+1]

valP ← PC+2

② 译码: valA ← R[rA] 读出A

valB ← R[rB] 读出B

③ 执行: valE ← valB + valA

Set CC

④ 访存: \

⑤ 写回: R[rB] ← valE

⑥ 更新PC: PC ← valP.

② rmmovq rA, D(rB)

取指	icode:ifun ← M_1[PC] rA:rB ← M_1[PC+1] valC ← M_8[PC+2] valC中存D的地址. valP ← PC+10
译码	valA ← R[rA] valB ← R[rB]
执行	valE ← valB + valC D+R[rB]
访存	M_8[valE] ← valA 把rA值存进去
写回	
更新PC	PC ← valP

把rA的值写入 D+R[rB] 地址

③ popq rA. / pushq rA

R[ ]: 寄存器堆
M[ ]: 对地址进行1个字节的读写

指令	push rA	pop rA
取指	icode: ifun ← M_1[PC] rA: rB ← M_1[PC+1] valP ← PC+2	icode: ifun ← M_1[PC] rA: rB ← M_1[PC+1] valP ← PC+2
	无valC	无valC
译码	valA ← R[rA]	valA ← R[%rsp] 找原栈顶
	valB ← R[%rsp] 找新栈顶的值	valB ← R[%rsp]
执行	valE ← valB+(-8)	valE ← valB+8 弹出后, 找原栈顶.
访存	M_8[valE] ← valA	valM ← M_8[valA]
写回	R[%rsp] ← valE	R[%rsp] ← valE R[rA] ← valM
更新 PC	PC ← valP	PC ← valP

push rA: 找新栈顶等位置
① R[%rsp] -= 8
② rA寄存器的值存到[rsp-8]的位置
valA

④ cmovXX rA rB. (这个可以不看)

	cmovXX rA, rB
取指	icode:ifun ← M_1[PC] rA:rB ← M_1[PC+1] valP ← PC+2
译码	valA ← R[rA] valB ← 0
执行	valE ← valB + valA If ! Cond(CC,ifun) rB ← 0xF
访存	
写回	R[rB] ← valE
更新PC	PC ← valP

条件成立则把rA的值传给rB

读A.

valE = A.

→ 若条件不成立: dstE 写为 0xF, 表示"不写寄存器"

→ 若条件成立: 把rA的值传给rB

⑤ jXX Dest

	jXX Dest
取指	icode:ifun ← M_1[PC] valC ← M_8[PC+1] 把Dest存入valC. valP ← PC+9
译码	
执行	Cnd ← Cond(CC,ifun) 是否成立?
访存	
写回	
更新PC	PC ← Cnd ? valC : valP

条件成立则跳到Dest.

不成立,继续执行下一条.(valP)

↓ 成立→Dest

→ 不成立 → valP.

⑥ Call Dest

	call Dest
取指	icode:ifun ← M_1[PC] valC ← M_8[PC+1] valP ← PC+9
译码	valB ← R[%rsp]
执行	valE ← valB + -8
访存	M_8[valE] ← valP
写回	R[%rsp] ← valE
PC更新	PC ← valC

{ 返回地址 valP 压入栈
  PC跳到 Dest

读栈指针.

栈指针减8.(在ALU)

返回地址入栈

更新栈指针.

PC跳到 Dest

⑦ ret

	ret
取指	icode:ifun ← M_1[PC]
译码	valA ← R[%rsp] valB ← R[%rsp]
执行	valE ← valB + 8
访存	valM ← M_8[valA]
写回	R[%rsp] ← valE
更新PC	PC ← valM

从栈中弹出返回地址
PC跳至这个返回地址.

读栈指针

读栈顶的小指针.

把返回地址存入valM

更新栈指针

PC跳至返回地址.

第五章 优化

1. [常见优化思路]

① 代码移动：把循环中不变的计算移到循环外，减少重复计算

```
double sum=0;
for(int i=0; i<n; i++){
    sum+= a[i]*cos(theta);
}
```

⇒

```
double sum=0, c=cos(theta);
for(int i=0; i<n; i++){
    sum+= a[i]*c;
}
```

② 消除不必要内存引用：用局部变量/寄存器暂存结果.

```
void sum(int *a, int n, int *dest){
    *dest=0;
    for(int i=0; i<n; i++){
        *dest += a[i];
    }
}
```

（*dest 反复读写）

⇒

```
void sum(int *a, int n, int *dest){
    int acc=0;
    for(int i=0; i<n; i++){
        acc+=a[i];
    }
    *dest = acc;
}
```

③ 强度削减：用简单操作（移位、加法）代替复杂操作（乘法、除法）

④ 循环展开：减少循环控制开销.

```
for(int i=0; i<n; i++){
    b[i]= a[i]+1;
}
```

⇒

```
int i
for(i=0; i+1<n; i+=2) {
    b[i]=a[i]+1;
    b[i+1]=a[i+1]+1;
}
for(; i<n; i++){
    b[i]=a[i]+1;
}
```

⑤ 多路累积：多个累加器减少串行依赖链

```
double sum(double *a, int n){
    double acc=0;
    for(int i=0; i<n; i++){
        acc+=a[i];
    }
    return acc;
}
```

每次都必须等一次的acc

```
double sum(double *a, int n){
    double acc=0, acc2=0;
    int i;
    for(i=0; i+1<n; i+=2){
        acc+=a[i];
        acc2+=a[i+1];
    }
    for(; i<n; i++){
        acc+=a[i];
    }
    return acc+acc2;
}
```

⑥ 重新结合：改变结合顺序，缩短关键路径

```
for(int i=0; i<n; i++){
    acc+=a[i];
}
```

```
for(int i=0; i+1<n; i+=2){
    acc+=(a[i]+a[i+1])
}
```

浮点数不符合结合律，不要用.

⑦ 矩阵优化：局部变量+循环展开+多路累加.

矩阵乘法

```
for(i=0; i<n; i++){
    for(j=0; j<n; j++){
        C[i][j]=0;
        for(k=0; k<n; k++){
            C[i][j]+= A[i][k]*B[k][j];
        }
    }
}
```

法1：局部变量+2路展开

```
for(i=0; i<n; i++){
    for(j=0; j<n; j++){
        int k;
        double acc0=0, acc1=0;
        for(k=0; k+1<n; k+=2){
            acc0+=A[i][k]*B[k][j];
            acc1+=A[i][k+1]*B[k+1][j];
        }
        for(; k<n; k++){
            acc0+=A[i][k]*B[k][j];
        }
        C[i][j]=acc0+acc1;
    }
}
```

→空间局部性(Ch6)更好.

法2：改变循环顺序+2路展开.

```
for(int i=0; i<n; i++){
    for(int j=0; j<n; j++){
        C[i][j]=0; //先清零
    }
}
for(int k=0; k<n; k++){
    for(int i=0; i<n; i++){
        double r=A[i][k]; //局部变量
        int j;
        for(j=0; j+1<n; j+=2){
            C[i][j]+=r*B[k][j]; //空间局部性.
```

```
            C[i][j+1] += r*B[k][j+1];//2次
        }
        for(;j<n;j++){
            C[i][j] += r*B[k][j];
        }
    }
}
```

2.[优化存在的问题]

6. 由于 C 语言提供指针运算，因此 _____ 现象会导致潜在的性能下降和正确性下降
   A. 非法指针
   B. 函数调用
   C. 内存别名 ✓
   D. 缺页

内存别名会阻碍优化.

第六章

1. [存储器层次]

寄存器
L1 Cache
L2 Cache
L3 Cache
主存DRAM
SSD/HDD
远程存储.

大小从小到大
价格从大到小.

1. 位于存储器层次结构中的最顶部的是( )。
A. 寄存器 B. 主存 C. 磁盘 D. 高速缓存

A.

2. [局部性]

6. 关于局部性的描述，不正确的是（C）
A. 循环通常具有很好的时间局部性
B. 循环通常具有很好的空间局部性
C. 数组通常具有很好的时间局部性 × 顺序扫一遍数组时，每个元素只访问一次.
D. 数组通常具有很好的空间局部性

对循环来说：
- 循环中的命令会被反复执行 → 时间局部性
- 循环体指令通常连续存放 → 空间局部性

3. [计算磁盘平均访问时间]

平均访问时间 = 平均寻道时间 + 平均旋转延迟 + 平均传送时间

平均旋转延迟 = 一圈旋转时间/2　　平均传送时间 = 一圈旋转时间/每磁道扇区数

1. 对于一个磁盘，其平均旋转速率是 15000 RPM，平均寻道时间是 4ms，单个磁道上平均扇区数量是 800，则这个磁盘的平均访问时间是 _____

15000 RPM (round per minute) = $\frac{15000}{60}$ = 250转/s

一圈旋转时间 $\frac{1}{250}$ = 4ms

⇒平均旋转延迟 4ms/2 = 2ms

4 + 2 + 0.005 = 6.005ms

平均传送时间 4ms/800 = 0.005ms.

4. [磁盘容量计算]

容量 = 盘面数 × 每盘面磁道数 × 每磁道扇区数 × 每扇区字节数

默认1个盘片2个面 (柱面数)

2. 对于一个磁盘，其有两个扇片，10000 个柱面，每个磁道平均有 400 个扇区，而每个扇区平均有 512 个字节。那么这个磁盘的的容量为 _____

2个扇片有4个盘面.

容量为 4×10000×400×512 = 8192000000 字节 8.192×10^9

= 8.192 GB

5. [Cache参数计算]

3. Cache 为 8 路的 2M 容量，B=64，则其 Cache 组的位数 s= 12

8路: E=8. B=64. C = S·E·B.

$$\therefore S = \frac{C}{E\cdot B} = \frac{2\times2^{20}}{8\times64} = 2^{12}$$

$s = \log_2 S = 12.$

6. [Cache地址划分]

CT CI CO

地址: t | s | b.

t = 物理地址 − s − b.

1. 对于一个机器而言，有如下的假设，内存是字节寻址，并且内存访问是 1 字节的字。地址宽度是 13 位，其高速缓存是 2 路组相联的，块大小是 4 字节，一共有 8 个组。高速缓存的具体内容如图所示。 S E B

a) 对于该地址格式进行划分，划分出 tag 位，组索引位和块内偏移的区间。

E=2. S=8. B=4

$e = \log_2 E = 1$, $s = \log_2 S = 3$ $b = \log_2 B = 2$

t = 13 − s − b = 8位.

地址划分(小端序):
- tag位: [12:5]
- 组索引: [4:2]
- 块内偏移位: [1:0]

7.[缓存命中的判断]

2路组相联高速缓存

组索引	行0 标记位	行0 有效位	行0 字节0	行0 字节1	行0 字节2	行0 字节3	行1 标记位	行1 有效位	行1 字节0	行1 字节1	行1 字节2	行1 字节3
0	09	1	86	30	3F	10	00	0	—	—	—	—
1	45	1	60	4F	E0	23	38	1	00	BC	0B	37
2	EB	0	—	—	—	—	0B	0	—	—	—	—
3	06	0	—	—	—	—	32	1	12	08	7B	AD
4	C7	1	06	78	07	C5	05	1	40	67	C2	3B
5	71	1	0B	DE	18	4B	6E	0	—	—	—	—
6	91	1	A0	B7	26	2D	F0	0	—	—	—	—
7	46	0	—	—	—	—	DE	1	12	C0	88	37

(接上题)

b) 对于地址 0x0E34，指出其对应的 tag 位，组索引位和块内偏移的值，并说明高速缓存是否命中，如果命中，写出对应的字节（用 16 进制表示）。

c) 对于地址 0x0DD5，指出其对应的 tag 位，组索引位和块内偏移的值，并说明高速缓存是否命中，如果命中，写出对应的字节（用 16 进制表示）。

b) 0x0E34 = 0 1110 0011 | 010 0
t S b

t=0x71 S=0x5 b=0x0

查上表：命中 0x0B.

c) 0x0DD5 = 0 1101 1101 | 010 | 1
t S b

t=0x6E S=0x5 b=0x1

有效位 valid=0，即不命中.

8.[步长为1的数组不命中率]

5. 若高速缓存的块大小为 B(B>8)字节，向量 v 的元素为 int，则对 v 的步长为 1 的应用的不命中率为_____

一个 int 为 4 bytes. 一个 block 可以放 $\frac{B}{4}$ 个 int

对于每个块，只有第1个 int 会出现不命中，之后的 int 均命中.

∴ 不命中率：$\frac{1}{\frac{B}{4}} = \frac{4}{B}$

9.[结构体数组的不命中率]

2. 在一台具有块大小 16 字节（B=16）、整个大小为 1024 字节的直接映射数据缓存的机器上测量如下代码的高速缓存性能：

假设：

- sizeof(int) = 4。
- grid 从内存地址 0 开始。
- 这个高速缓存开始时是空的。
- 唯一的内存访问是对数组 grid 的元素的访问，变量 i、j、total_x 和 total_y 存放在寄存器中。
- 数据结构定义

```
struct position {
    int x;
    int y;
};

struct position grid[16][16];

int total_x = 0, total_y = 0;
int i, j;
```

A. Test 1

```
for (i = 0; i < 16; i++){
    for(j = 0; j < 16; j++){
        total_x += grid[i][j].x;
    }
}

for (i = 0; i < 16; i++){
    for(j = 0; j < 16; j++){
        total_y += grid[i][j].y;
    }}
```

16×16

16×16

1. 读总数是多少？
2. 缓存不命中的读总数是多少？
3. 不命中率是多少？

B. Test 2

```
for (i = 0; i < 16; i++){
    for(j = 0; j < 16; j++){
        total_x += grid[i][j].x;
        total_y += grid[i][j].y;
    }}
```

4. 读总数是多少？
5. 缓存不命中的读总数是多少？
6. 不命中率是多少？
7. 如果高速缓存有两倍大，那么不命中率是多少？

对一个16字节块：存
grid[0][0].x, grid[0][0].y,
grid[0][1].x, grid[0][1].y.

结构体大小：4+4=8bytes

grid数组大小：16×16×8=2048B

占用Cache块：2048B/16B=128个

对于Cache：1024B/16B=64行

（Cache一次只能容纳两半个grid，block编号k(0~127)被映射到k%64）

对Test1：

1. 读总数：16×16+16×16=512
2. 对第1个循环：

读grid[i][0].x 会不命中
grid[i][1].x 命中
grid[i][2].x 会不命中
⋮

第2个循环时，后半个grid刚好把前半个挤出去，所以也有一半会不命中。
不命中总数 256.

3. 由前面分析：

不命中率为50%.

本质是容量不命中.

对Test2：

4. 读总数 16×16×2=512
5. 6. 读 grid[i][0].x 会不命中
grid[i][0].y 命中
grid[i][1].x 命中
grid[i][1].y 命中

故不命中率为25%。

7. Test2不存在冲突不命中，容量不命中，

故不命中率不变。

10. [三种不命中]

- 冷不命中：第一次访问某块，Cache内不存在有
- 容量不命中：工作集太大，Cache总容量装不下（通常可通过增大Cache容量缓解）
- 冲突不命中：Cache总容量够，但多个块映射到同一组，互相挤掉。

11. [Cache地址排序的原因]

3. 对于一个地址，高速缓存通常使用地址的中间部分作为组索引，为什么不用高位地址作为组索引？

低位作为块内偏移，同一块内的连续字节共享一个Cache line。

高位在连续访问时很慢（空间局部性）。

用中间位作组索引可以让连续内存块分布到不同组，提高Cache利用率，减少冲突不命中。

12. [SSD]

SSD固态硬盘 随机读 (远远) >> 机械硬盘

随机写与读性能不一致

SSD随机读总体优于机械硬盘。

第9章

1. [虚拟地址空间]

9. 关于虚拟地址空间的叙述，不正确的是 _____
A. DRAM 空间可以理解为虚拟地址的缓存 ✓
B. 每个程序都有独立的虚拟地址空间 ✓
C. 虚拟地址直接映射到物理地址的连续区域 ×
D. 虚拟内存增强了进程的局部性 ✓

虚拟地址到物理地址通过页表映射，不要求连续.

2. [虚拟页面的状态]

8. 虚拟页面的状态有 未分配、已缓存、未缓存共 3 种。

虚拟页未与任何数据关联.

虚拟页在DRAM中

虚拟页在磁盘上，不在DRAM中

3. [PTE数量计算]

6. 某 CPU 使用 32 位虚拟地址和 4KB 大小的页时，需要 PTE 的数量是 _____（不考虑多级页表情况）

(VPN位数)

页大小：$4KB = 2^2 \times 2^{10} B = 2^{12} B$

⇒ VPO位数：$\log_2 2^{12} = 12$

VPN位数：$32 - 12 = 20$ ⇒ 虚拟页数为 2^{20}.

一级页表，一个虚拟页 = 一个PTE

∴ PTE数为 2^{20}.

4. [地址翻译与Cache]

已知：虚拟地址14位，物理地址12位，页大小64B，TLB 16个字组，4路组相联. Cache直接映射，16组，每块4B.

请翻译虚拟地址 VA=0x03D4，回答：

① VPN=？ ② VPO=？ ③ TLB是否命中？ ④是否缺页？

⑤ PPN=？ ⑥ PA=？ ⑦ Cache的 CT. CI. CO=？ ⑧ Cache是否命中？

(拆物理地址PA) t s b.

PPO → 页大小 2^P B

VA = VPN | VPO

PA = PPN | PPO

⑨ 取出的字节是多少？

VPN = TLBT | TLBI

t位

↳ TLB组数 2^i = TLB中总数 / 路数.

组	标记	PPN	有效位	标记	PPN	有效位	标记	PPN	有效位	标记	PPN	有效位
0	03	–	0	09	0D	1	00	–	0	07	02	1
1	03	2D	1	02	–	0	04	–	0	0A	–	0
2	02	–	0	08	–	0	06	–	0	03	–	0
3	07	–	0	03	0D	1	0A	34	1	02	–	0

索引	标记位	有效位	块0	块1	块2	块3
0	19	1	99	11	23	11
1	15	0	–	–	–	–
2	1B	1	00	02	04	08
3	36	0	–	–	–	–
4	32	1	43	6D	8F	09
5	0D	1	36	72	F0	1D
6	31	0	–	–	–	–
7	16	1	11	C2	DF	03

Idx	Tag	Valid	B0	B1	B2	B3
8	24	1	3A	00	51	89
9	2D	0	–	–	–	–
A	2D	1	93	15	DA	3B
B	0B	0	–	–	–	–
C	12	0	–	–	–	–
D	16	1	04	96	34	15
E	13	1	83	77	1B	D3
F	14	0	–	–	–	–

①② 页大小 $64B = 2^6 B$ ⇒ VPO = PPO = 6位

VPN = 14 − 6 = 8位, PPN = 12 − 6 = 6位.

∴ 0x03D4 ⇒ 0x0000001111010100

VPN=0x0F VPO=0x14

③ TLB组数: $\frac{16}{4} = 4$

TLBI = $\log_2 4 = 2$.

TLBT = 8 − 2 = 6.

TLBT TLBI

00001111

⇒ TLBT = 0x03

TLBI = 0x3

⑤ ⇒ 命中, PPN = 0x0D

④ 有命中肯定不缺页.

⑥ PA = PPN | PPO = 001101010100 = 0x354.

⑦ Cache: 16组: S=16 $s = \log_2 S = 4$ = CI

B=4 $b = \log_2 B = 2$. = CO

(4byte/tag)

t = 12 − 4 − 2 = 6 = CT

t s b

PA = 001101010100

t=0x0D S=0x5 b=0x0

⑧⑨ 查表: 取出0x36.

valid=1 命中.

5.[地址翻译与Cache]

六、综合分析题（共 15 分，每空 1 分）

某计算机系统的情况如下：

1. 内存是按字节寻址，字长为 2 字节（非 4 字节），采用小端模式存储；
2. 虚拟地址长度：24 位；
3. 使用一级页表，页面大小是 2048 字节；
4. 物理地址的位数是 19 位；
5. TLB（翻译后备缓冲器）是 4 路组相连，共 16 个条目；
6. L1 d-Cache 是物理寻址、直接映射（每组一行）、行大小 8 字节，共 8 个组；
7. TLB、Cache 的当前数值如表 1、2 所示。

B　S

(1) 页面大小 2048B = 2^{11}B

⇒ VPO, PPO 位数为 11 位

VPN = 24 − 11 = 13 位

PPN = 19 − 11 = 8 位

PTE = 页面数 = 2^{13}

(2) TLB 组数 = 4

TLBI = $\log_2 4$ = 2 位

TLBT = 13 − 2 = 11 位

表 1：TLB 数值表

组	标记位	PPN	有效位	标记位	PPN	有效位	标记位	PPN	有效位	标记位	PPN	有效位
0	0CD	09	1	AA1	00	1	3E0	62	1	C4C	48	1
1	312	45	0	010	75	1	987	3A	1	D39	3F	0
2	038	E3	0	0A7	13	0	18B	52	1	49B	11	0
3	6C0	42	0	075	50	0	013	39	1	0F2	0D	0

(3) 行大小 8B.

∴ b = $\log_2 8$ = 3 位

S = $\log_2 8$ = 3 位

t = 19 − 3 − 3 = 13 位

表 2：L1 d-Cache 的数值

索引	标记位	有效位	块 0	块 1	块 2	块 3	块 4	块 5	块 6	块 7
0	35E	1	42	A0	75	50	42	00	05	50
1	27B	1	08	E3	00	A7	13	00	88	52
2	C54	1	3F	75	AB	11	25	78	9A	00
4	A32	1	97	3A	91	D3	3F	12	86	22
5	C30	1	30	62	15	4C	48	A1	12	5C
6	B26	1	01	25	3E	62	1F	C4	85	12
7	01A	1	98	3A	12	D3	3F	3C	4D	5E

(1) VPN 的位数是 13 位，页表项 PTE 的总数量是 2^{13} 个。

(2) TLB 中，组索引是 2 位，标记 Tag 的位数是 11 位。

(3) 在 L1 d-Cache 中，块偏移的位数是 3 位，组索引的位数是 3 位，标记的位数是 13 位。

(4) CPU 从虚拟地址 0x7C0515 读取一个字的数值，将虚拟地址翻译成物理地址，并获取数值的过程中，写出下表中各参数对应的数值（8 分，每空 1 分）。

VA 0111 1100 0000 0101 0000 1010 1

VPN = TLBT + TLBI　VPO

VPN = 0x0F80

TLBI = 0x0

TLBT = 0x3E0

PA = 0110 0010 1010 0010 1010 1

t　s　b

= 0x31515

t = 0x0C54

s = 0x2

b = 0x5

参数	数值
虚拟地址	0x7C0515
虚拟页号 VPN	0x0F80
TLB 索引	0x0
TLB 标签 Tag	0x3E0
TLB 命中？(Y/N)	Y
物理页号 PPN	0x62
物理地址	0x31515
Cache 命中？(Y/N)	Y
读取的数值	0x78

从块5处开始，读两字节，小端序。

0x9A78

6. [缺页异常]

虚拟内存发生缺页时，缺页中断由MMU触发。

当处理程序处理完缺页错误后，会重新执行引发缺页的指令。

7. [内存映射]

Linux虚拟内存区域可以映射到普通文件和匿名文件，这两种类型的对象中的一种。

第1章

计算机系统的几种抽象：

- **指令集架构** 提供了对实际处理器硬件的抽象。
- **文件** 是对I/O设备的抽象。
- **虚拟内存** 是对主存和硬盘的抽象。
- **进程** 是对一个正在运行的程序（或处理器、主存和IO设备）的抽象。
- **虚拟机** 是对整个计算机的抽象，包括操作系统、处理器和程序。